```
import math, numpy, pyglet, obstacle, guy, res, random
from copy import copy
from scipy import *

game_window = pyglet.window.Window(900, 750)
white = (255,255,255,255)
pyglet.gl.glClearColor(*white)
main_batch = pyglet.graphics.Batch()

#coordinate of the goal and obstacle
x_coord = [120,300,630,250,100,406,700,500,384,820]
y_coord = [100,100,430,250,400,350,720,250,50,720]
coord = zip(x_coord,y_coord)

goal = coord[-1]
target = pyglet.sprite.Sprite(img=res.goal,x=goal[0], y=goal[1], batch=main_batch)

obs = [pyglet.sprite.Sprite(img=res.obstacle,x=x_coord[i], y=y_coord[i], batch=main_batch) for i in range(1,len(x_coord)-1)]

objet = pyglet.sprite.Sprite(img=res.player,x=x_coord[0], y=y_coord[0], batch=main_batch)


def dist(a,b):
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)


@game_window.event #tells the window to draw objects
def on_draw():
    game_window.clear()
    main_batch.draw()

extlen = 10
def ef(src, target):
  global extlen
   #extend as far as possible without colliding

  A = 50


  direction = [goal[i]-src[i] for i in xrange(len(src))]
  normdir = [direction[i]/dist([0,0], direction) for i in [0,1]]
  return [src[0] + extlen*normdir[0], src[1] + extlen*normdir[1]], normdir

def mdit_obs(obs,newpt):
  mdist = min(dist(newpt,[obs1.x,obs1.y]) for obs1 in obs)
  for obs1 in obs :
    if dist(newpt,[obs1.x,obs1.y])==mdist :
      nearobs=[obs1.x,obs1.y]
  return mdist , nearobs

def ef1(src, nearobs):
  global extlen
  d=dist(scr,nearobs)
  normdir=[-(scr[1]-nearobs[1])/d,(scr[0]-nearobs[0])/d]
  return [src[0] + extlen*normdir[0], src[1] + extlen*normdir[1]], normdir

scr=coord[0]
```

```python
d=dist(scr,goal)
normdir=[(scr[0]-goal[0])/d,(scr[1]-goal[1])/d]
def update(dt):
  global scr, objet, target, coord , obs , normdir
  #while dist((objet.x, objet.y), goal)<=10:
  newpt,normdir=ef(scr, target)
  mdist,nearobs = mdit_obs(obs,newpt)
  if mdist > 65:
    objet.rotation = -270*math.atan2(normdir[0],normdir[1])/math.pi
    objet.x, objet.y=newpt[0],newpt[1]
    scr=[objet.x, objet.y]
  else :
    newpt,normdir=ef1([objet.x, objet.y], nearobs)
    objet.rotation = 100*math.atan2(normdir[0],normdir[1])/math.pi
    objet.x, objet.y=newpt[0],newpt[1]
    scr=[objet.x, objet.y]
  if dist(scr, goal)<=80:
    return


if __name__ == "__main__":
    # Update the game 120 times per second
    pyglet.clock.schedule_interval(update, 1/5.0)
    #pyglet.clock.schedule_interval(dispAI, 0.2)
    # Tell pyglet to do its thing
    pyglet.app.run()
```

robot.py